SEMANA 7

Programación de Entornos Multimedia

Vistiendo el Mundo: Importación de Assets, HDRI y MatCaps.

Docente: Mg. Gabriel Guerrero Arellano

De matemáticas, físicas y cubos grises a modelos 3D profesionales e iluminación con calidad fotorrealista.



¿Dónde estamos en la Ruta de Aprendizaje?

Semana 6 · El Cerebro

Físicas Avanzadas y Control. Nuestro cubo tiene masa, detecta colisiones y responde al teclado del usuario.

Semana 7 · La Piel

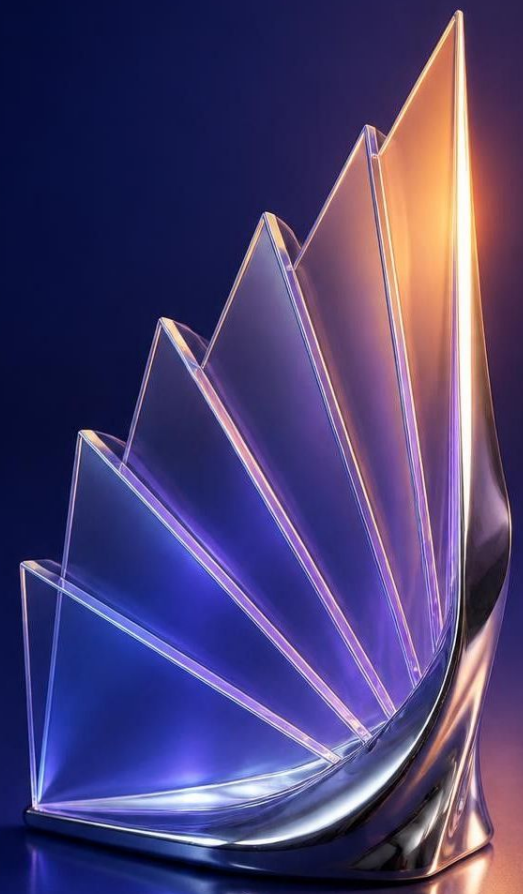
El cubo funciona, pero no es atractivo. Un Asset externo será su disfraz profesional: un carro, una nave o un personaje animado.

Más allá de la Geometría Básica

Un **Asset** es un recurso externo creado por un artista: modelos 3D, texturas o audios que importamos a nuestro código.

Cada Megabyte cuenta

El reto de la Web es verse bien y **descargarse rápido**. No podemos usar los mismos formatos que Hollywood: necesitamos optimización extrema.



El “JPEG del 3D”
Formato glTF / .glb

Antes · .OBJ / .FBX

Exportar a la web era un dolor de cabeza con formatos pesados.

Hoy · glTF

GL Transmission Format: estándar abierto diseñado para que los navegadores lo lean rápido.

Usaremos **.glb**, la versión binaria: empaqueta geometría, texturas y animaciones en un solo archivo, como un .zip súper comprimido.

¿Por qué glTF es tan rápido?

Imagina comprar un Armario de ropa.



Formatos Viejos · .FBX

Meter un Armario ya Armado: ocupa todo el camión de mudanza, es pesado y cuesta meterlo por la puerta.



Formato .glb · IKEA

Desarmado y empaquetado al vacío en cajas planas: pesa muy poco. El navegador lo recibe y el procesador lo “arma” en milisegundos, directamente en la memoria de la tarjeta gráfica.

Capturando la luz del mundo real

IBL (Iluminación Basada en Imágenes) usa una fotografía panorámica de un lugar real para lograr reflejos 100% realistas.

Usamos **HDRI (High Dynamic Range Imaging)** o Environment Maps.

JPG normal	HDRI
Solo guarda color.	Guarda intensidad lumínica matemática.

El sol emite miles de veces más energía que el cielo azul. Three.js usa esa energía para iluminar el modelo.





Entendiendo el HDRI

Imagina a un actor en un teatro oscuro.

En vez de 50 linternas para simular un bosque, muy costosas para el procesador, lo colocamos dentro de una **pantalla LED circular de 360 grados** que muestra un bosque brillante.

La luz de la pantalla lo ilumina perfectamente. **¡Eso es el HDRI!**

Magia visual a costo cero (MatCaps)

HDRI consume recursos. Para celulares de gama baja usamos **MatCaps (Material Capture)**.

Materiales PBR (**Physically Based Rendering** (Renderizado Basado en Físicas)

Una cara limpia con luces de estudio reales: alto consumo. es el estándar de la industria que simula matemáticamente cómo la luz del mundo real interactúa con las superficies (calculando valores como qué tan metálico o rugoso es un objeto). En lugar de "pintar" o fingir los brillos, el motor gráfico calcula la física real de la luz rebotando en el material.

TAl tener que calcular cómo la luz incide, rebota y genera sombras en tiempo real para lograr fotorrealismo, esta técnica exige un **alto consumo** de recursos de procesamiento de la tarjeta gráfica (GPU).

MatCap · Maquillaje

Una imagen plana con sombras oscuras y brillos claros “finge” luces. El motor no calcula ninguna luz: envuelve el modelo con esa imagen.

¡Un truco óptico ultra ligero!

Nuestro ecosistema para el Laboratorio

Para la práctica de hoy: nuevas herramientas para nuestro código.

GLTFLoader

Sabe “desarmar” la caja de IKEA (.glb).

RGBELoader

Lee los datos de luz de un archivo .hdr.

TextureLoader

Carga nuestros MatCaps.

Archivos Base

Modelos y texturas de PolyHaven listos para integrar.

Reglas para un Laboratorio sin errores

1 Rutas Absolutas

Todo archivo externo (.glb, .hdr, .png) DEBE estar en `public` de Vite. La ruta empieza con /: `/models/carro.glb`.

---

2 La Consola es tu amiga

Pantalla negra: abre `F12`. “404 Not Found” significa que escribiste mal el nombre del archivo.

---

3 Asincronismo

Los modelos pesan. El código no se detiene: se cargan en segundo plano mediante callbacks.

Paso 0: Creación del Proyecto con Vite

Antes de escribir una sola línea de código, necesitamos construir los cimientos de nuestro entorno web usando la terminal de VS Code. Sigan estos comandos en orden:



1. Crea el proyecto Vite

Ejecuten `npm create vite@latest .` **¡No olviden el punto "."!** Así se instala en esta misma carpeta. Seleccionen **Vanilla** y **JavaScript**. (si la pantalla se encuentra en congelada dar enter y continua)



2. Instala las dependencias base

Ejecuten `npm install` para descargar los cimientos de Node.js.



3. Instala Three.js

Nuestro motor 3D.
Ejecuten `npm install three`.



4. ¡Enciende el motor!

Ejecuten `npm run dev`. Abran el enlace local (ej. `http://localhost:5173`) para ver los cambios en vivo.

Paso 1: Encendiendo el motor y limpiando la casa

Para evitar "pantallas blancas", debemos limpiar la plantilla de Vite.

1. En la terminal de VS Code ejecuta `npm run dev` para levantar el servidor.
2. Abre `index.html`, borra todo su contenido y asegúrate de incluir el lienzo donde Three.js va a dibujar.

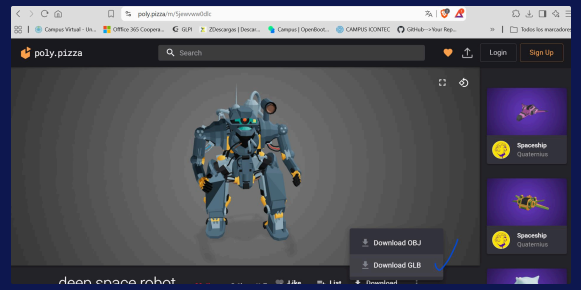
Código HTML:

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <style>body { margin: 0; overflow: hidden; }</style>
 </head>
 <body>
  <!-- Este es nuestro lienzo en blanco -->
  <canvas id="bg"></canvas>
  <script type="module" src="/src/main.js"></script>
 </body>
</html>
```

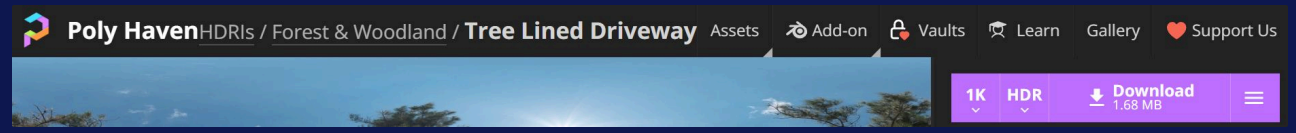
Paso 2: La Estructura Pública de Archivos

Three.js es muy estricto con las rutas. Si no encuentra el archivo, arrojará un error 404.

1. En el panel izquierdo, busca la carpeta `public/` (Vite expone esto a la web).
2. Dentro de `public`, haz clic derecho y crea dos carpetas: `models` y `textures` "EN MINUSCULAS"
3. Descarga un modelo gratuito (ej. `robot.glb`) y arrástralo a la carpeta `models`. por ejemplo del sitio: **https://poly.pizza/**



1. Descarga un mapa de luces (ej. `bosque.hdr`) por ejemplo desde el sitio **https://polyhaven.com/** se recomienda filtrar de la siguiente manera **Resolución 1k y formato HDR** para no generar lag en el servidor y correr con agilidad:



y arrástralo a la carpeta `textures` como bosque.hdr

Esquema de carpetas

```
proyecto/
└── public/
    ├── models/
    │   └── robot.glb
    └── textures/
        └── bosque.hdr
```

Paso 3: Construyendo el Set de Grabación

(Abre el archivo src/main.js, borra TODO el texto que tenga y escribe esto) No podemos meter un actor si no hay un set, una cámara y alguien grabando.

```
import * as THREE from 'three';
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js';
import { RGBELoader } from 'three/examples/jsm/loaders/RGBELoader.js';
// 1. Configuramos Escena, Cámara y Renderizador
const canvas = document.querySelector('#bg');
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 1000);
camera.position.z = 5; // Alejamos la cámara para poder ver
const renderer = new THREE.WebGLRenderer({ canvas: canvas });
renderer.setSize(window.innerWidth, window.innerHeight);
```

Paso 4: Cargando el Cielo Fotográfico (HDRI)

(Continuamos en `main.js`) Usaremos `RGBELoader` para leer nuestro archivo de rango dinámico y envolver nuestra escena con luz realista.

```
// 2. Cargamos el mapa de iluminación (HDRI)
const rgbeLoader = new RGBELoader();
rgbeLoader.load('/textures/bosque.hdr', (environmentMap) => {
    // Configuramos la imagen para que sea una esfera 360°
    environmentMap.mapping = THREE.EquirectangularReflectionMapping;
    // Lo aplicamos como fondo visual y como luz global
    scene.background = environmentMap;
    scene.environment = environmentMap;
});
```

Paso 5: Cargando el Modelo 3D (.glb)

(Continuamos en `main.js`) Usamos `GLTFLoader` para "desempaquetar" el archivo. Es asíncrono, por lo que el modelo aparecerá cuando termine de descargar.

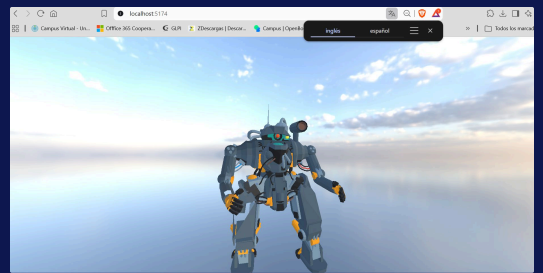
```
// 3. Cargamos el modelo 3D
const gltfLoader = new GLTFLoader();
let miPersonaje = null; // Variable global para poder animarlo luego
gltfLoader.load('/models/robot.glb', (gltf) => {
    miPersonaje = gltf.scene; // Extraemos la geometría
    // Ajustamos tamaño y lo colocamos en la escena
    miPersonaje.scale.set(3, 3, 3);
    miPersonaje.position.y = -1;
    scene.add(miPersonaje);
});
```

Paso 6: El Bucle de Animación

(Al final de main.js) Si no le decimos a Three.js que tome "fotos" de la escena 60 veces por segundo, la pantalla se quedará paralizada.

```
// 4. Bucle de Animación
function animate() {
    requestAnimationFrame(animate);
    // Si el modelo ya cargó, lo hacemos rotar
    if (miPersonaje) {
        miPersonaje.rotation.y += 0.01;
    }
    // Tomamos la foto y la mostramos en pantalla
    renderer.render(scene, camera);
}
// ¡Iniciamos el bucle!
animate();
```

Punto de Control: Guarda los archivos. ¡Tu modelo ya debe verse en pantalla rotando de forma fotorrealista!



Punto de Control en Vivo

Abre la pestaña de tu navegador donde corre tu servidor Vite.

1

¿El fondo se ve como una fotografía panorámica?

2

¿Aparece su modelo 3D?

3

Presionen las teclas de movimiento: ¿El modelo 3D se desplaza y colisiona con el piso?

⚠ Si algo falla, presionen `F12`. Revisen que los nombres de sus carpetas y archivos coincidan exactamente con lo que escribieron en `main.js`.

BONUS

Bonus 1: ¡Explora tu mundo 3D! (OrbitControls)

Hasta ahora somos simples espectadores. ¡Vamos a darle el control al usuario! Con `OrbitControls` podremos rotar, hacer zoom y desplazarnos por el entorno usando el mouse, tal como en un motor de videojuegos.

1. Importamos la herramienta

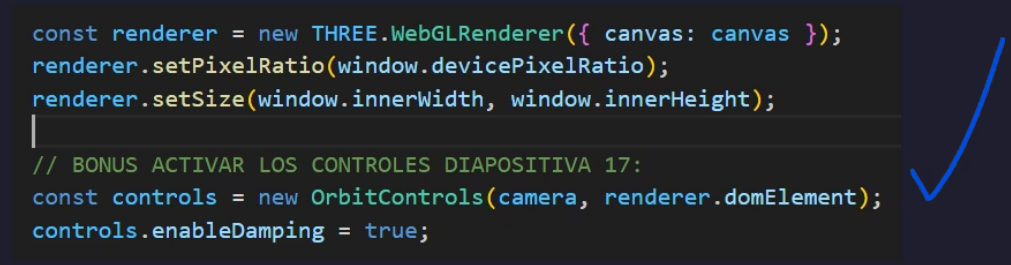
Sube a la primera línea de tu `main.js` y agrega esto junto a los otros imports.

```
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js';
```

2. Activamos los controles

Pega esto justo debajo de donde creaste tu `renderer`.

```
// Añadimos la cámara libre con efecto de inercia (suavizado)
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
```



BONUS

Bonus 2: Levitación y el Bucle Final

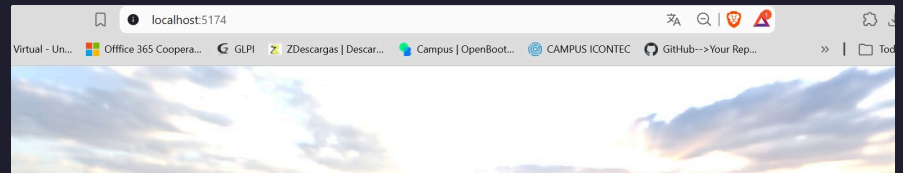
¿Y si hacemos que el robot parezca estar flotando o respirando? Usando una simple función matemática (Seno), podemos crear un efecto de levitación súper profesional.

3. Actualizamos el bucle de animación

Reemplaza tu función `animate()` actual por esta versión final.

```
function animate() {
    requestAnimationFrame(animate);
    if (miPersonaje) {
        // 1. Rotación constante
        miPersonaje.rotation.y += 0.01;
        // 2. ¡Efecto de levitación usando matemáticas!
        // Date.now() crea el tiempo, Math.sin hace que suba y baje suavemente
        miPersonaje.position.y = Math.sin(Date.now() * 0.002) * 0.5 - 1;
    }
    // 3. Actualizamos la inercia de la cámara en cada frame
    controls.update();
    // 4. Tomamos la foto y la mostramos
    renderer.render(scene, camera);
}
```

- activamos la rotación con el **clic izquierdo rpesionado**, se movera alrededor del robot el cual esta con una animación de flote:
-
- con clic derecho presionado ubicaremos libremente la camara.
-
- con el scroll + o scroll + alejamos o acercamos la camara



Tips Pro para Entornos Multimedia

Compresión DRACO

En proyectos reales, los .glb se comprimen con Draco, un algoritmo de Google.

20MB → 2MB

Resolución de Texturas

Una textura 4K arruinará la experiencia web. Para móviles, máximo **1024x1024** o **2048x2048**.

Limitar HDRIs

Pueden pesar muchísimo. Herramientas web los comprimen conservando los datos de luz importantes, sin pérdida visual notable.

Resultado Esperado de la Sesión

Al finalizar estas **3 horas**, deben mostrar a la clase:

1

Asset cargado

Proyecto Vite cargando un .glb desde /models.

2

Entorno iluminado

Iluminación global con un .hdr (Environment Map).

3

Movimiento sincronizado

Controles del teclado y físicas de la semana 6.

4

Opcional / Bonus

Un MatCap en un elemento secundario de la escena, rotación y camara libre con animación en el modelo importado.

Conclusiones y Preparación para la Semana 8

¿Qué aprendimos hoy?

Nuestro entorno deja de verse como un prototipo y empieza a verse como un producto real.

Estructura de carpetas `public/`, servidor local `npm run dev`, IBL y MatCaps: en la web, el rendimiento es tan importante como la estética.

Semana 8 · Integración con Blender

Hoy importamos un solo objeto. La próxima semana exportaremos un escenario entero de golpe y lo cargaremos en nuestro código con un solo archivo.

Prepararemos el ensamblaje de nuestro **Mundo Interactivo final**.